Parallel Processing of Large Weather Datasets Using OpenMP

A Sequential vs. Parallel Study of Weather Statistics: Performance, Scalability and Bottlenecks

Department of Information Technology

MKSSS's Cummins College of Engineering for Women, Pune

Academic Year 2026–2027

Course: 23PCIT501 Design and Analysis of Algorithms

Parallel Programming Project (T1)

Division: TY IT A

Submitted by

Akshita Pahade – UIT2024846

Shalaka Parhad– UIT2024847

Parul Deshmukh– UIT2024848

Samruddhi Pashankar – UIT2024849

1. Introduction

1.1 What is weather data processing?

A weather station logs readings such as temperature, humidity, pressure, wind speed and rainfall at regular intervals. After a few months the log has thousands of rows, and after a few years of many stations it has millions. Nobody reads these rows one by one. What people need are summary values: the average temperature of the period, the lowest pressure, the highest wind speed, the total rainfall. Getting these values out of the raw log is what we call weather data processing in this project.

1.2 Why parallel computing is useful

The statistics we want are sums, minimums and maximums. The order in which the records are added does not change the result. So the records can be divided into groups, each group can be summarised separately and at the same time, and the group summaries can be combined at the end. If p cores share the work equally, the computing step should take about 1/p of the sequential time. How much this helps the whole program depends on how much of the program is that computing step.

OpenMP (Open Multi-Processing) is a standard API for shared-memory parallel programming in C. The programmer adds compiler directives such as `#pragma omp parallel` to ordinary code, and the compiler and runtime create and manage the threads. All threads see the same memory, so the weather arrays do not have to be copied or sent anywhere.

2. Problem Statement

The project is to develop a computational system for processing large weather datasets and extracting useful information from historical or simulated observations, and to investigate its scalability for increasing dataset sizes.

Formal definition. Given N weather observations with values x_i for each of five variables (temperature, humidity, pressure, rainfall and wind speed), compute for every variable the sum, the mean $\mu = (1/N)\ \Sigma\ x_i$, the minimum and the maximum. Rainfall is reported as a total as well as a maximum.

3. System Requirements

Item	Details
Language	C with OpenMP
Source files	sequential.c (328 lines), parallel.c (581 lines); dashboard: index.html, app.js, styles.css
Timers	`clock()` in sequential.c and `omp_get_wtime()` in parallel.c, around the computation loop only

Table 1: Software and hardware used

4. Dataset Description

The experiments use a real dataset from an automatic weather station (AWS Dataset Pangandaran.csv, recorded by a PCE-FWS 20N instrument). Each row is one observation and has multiple columns.

Property	Value
Source	AWS Dataset Pangandaran.csv (real station data)
Valid observations	8,085 records
Period	28 March 2022 to 11 April 2023
File size	462 KB

Table 2: Dataset properties

The five measured variables, and the values our programs compute for them, are listed below. The same values are used later for the correctness check.

Variable	Unit	Statistic	Minimum	Average / Total	Maximum
Temperature	°C	mean, min, max	20.80	25.55	33.00
Humidity	%	mean, min, max	52.00	84.06	99.00
Relative pressure	hPa	mean, min, max	1005.70	1011.82	1017.60
Wind speed	km/h	mean, min, max	0.00	4.40	23.40
Rainfall (hourly)	mm	total, min, max	0.00	4758.90 (total)	38.10

Table 3: Variables processed and the results for the full dataset

5. System Architecture / Workflow

Both programs follow the same pipeline. The file is read once, the five numeric columns are stored in arrays, and only then the timed computation starts. Keeping the file reading outside the timer makes it possible to measure the speed of the computation alone.

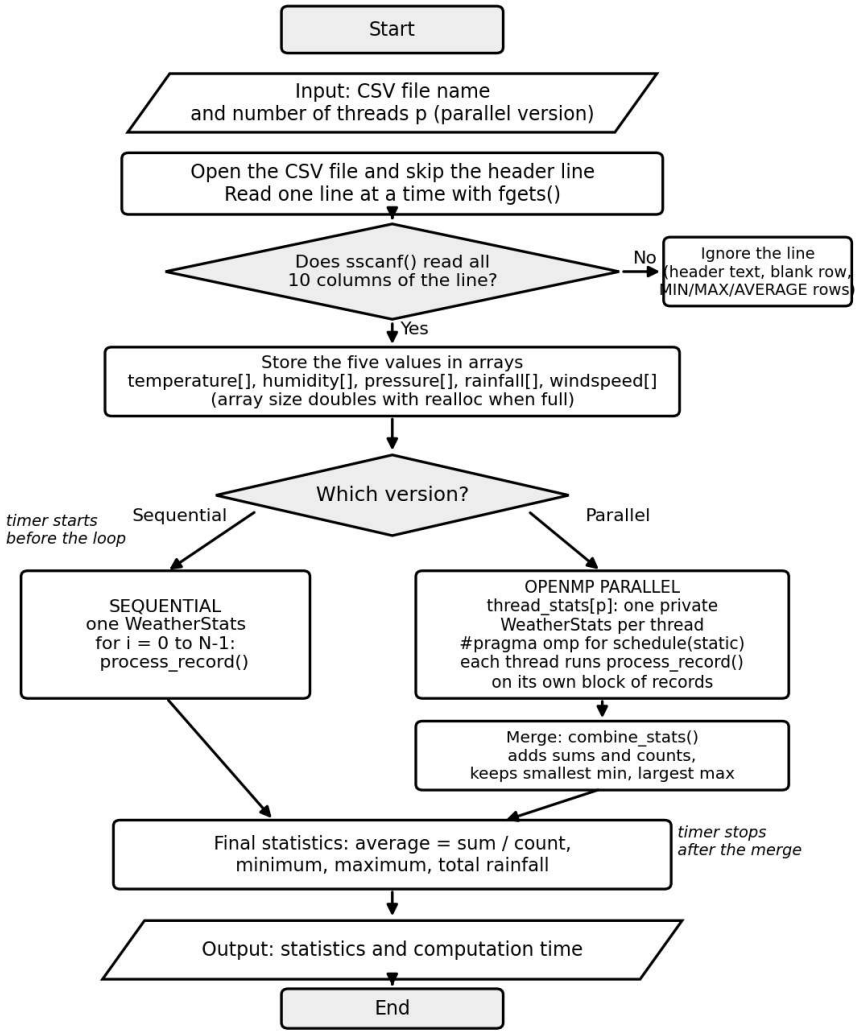


Figure 1: Flowchart of the weather processing system

Stage	What the code does
1. Read	The file is opened and read line by line with `fgets()`. The first line is skipped as the header.
2. Parse	`sscanf()` reads the ten columns of a line. Lines that do not give ten values (rest of the header, blank row, MIN/MAX/AVERAGE rows) are dropped.
3. Store	The values are saved in arrays.
4. Compute	`process_record()` updates the count, and the sum, minimum and maximum of every variable. In the sequential program one loop does this for all records. In the parallel program the same loop is shared by the threads.
5. Merge	Parallel version only: `combine_stats()` merges the private results of the threads.

Table 4: Processing stages and the code that does each of them

6. Methodology / Algorithm

6.1 The idea: a running summary

The program keeps a small structure called `WeatherStats`. It holds a count of records and, for each of the variables, a running sum, a running minimum and a running maximum. That makes 16 numbers in total. Before the first record, the sums are 0, every minimum is set to the largest possible double and every maximum to the smallest possible double, so that the very first record replaces them.

For every record the program adds the value to the sum and compares it with the current minimum and maximum. After the last record, the mean of a variable is its sum divided by the count. Nothing else is needed, so the whole dataset is handled in one pass and no sorting is required.

6.2 Why the work can be split

Suppose the records are cut into two halves and each half is summarised on its own. The two summaries can be joined into the summary of the full data by simple rules: add the counts, add the sums, take the smaller of the two minimums and take the larger of the two maximums. The result is the same as one pass over everything, whatever the order. This is the property that allows several threads to work on different parts of the data and merge their results later.

Quantity	Rule used to merge two summaries A and B
count	$count_A + count_B$
sum	$sum_A + sum_B$
minimum	smaller of min_A and min_B
maximum	larger of max_A and max_B
mean	merged sum / merged count (computed once, at the very end)

Table 5: Rules for merging partial results

6.3 Sequential algorithm

```
initialize S                    // sums = 0, min = +large, max = -large
for i = 0 to N-1:               // one pass over the records
S.count = S.count + 1
for each variable v:
S.sum[v] = S.sum[v] + x[v][i]
S.min[v] = smaller(S.min[v], x[v][i])
S.max[v] = larger(S.max[v], x[v][i])
mean[v] = S.sum[v] / S.count    // for every variable v
```

The loop touches every record exactly once, so the work is proportional to N. This version has no OpenMP directive. It is the reference for correctness and the baseline for timing.

6.4 Parallel algorithm

```
start p threads
each thread t creates its own summary S_t (same starting values as above)
parallel for i = 0 to N-1       // static schedule
   S_(thread of i) = update(S_(thread of i), record i)
wait until all threads are finished   // implicit barrier
F = new summary
for t = 0 to p-1:  F = merge(F, S_t) // rules of Table 5
```

With `schedule(static)` the records are divided into p blocks of nearly equal size. Thread t gets the t-th block of consecutive records, and when N is not a multiple of p, a few threads get one record more. Every thread only touches its own summary S_t, so threads never write to the same place. The merge at the end handles p small structures, so its cost depends on p and not on N.

7. OpenMP Parallelization

7.1 Choice of parallel programming model

The five arrays are about 40 bytes per record, which is only 323 KB for our dataset, and they are only read during the computation. A shared-memory model is therefore the natural choice. OpenMP can parallelise the existing loop with a few lines, and nothing has to be copied between threads.

Model	Strength	Limitation	Fit for this project
OpenMP (chosen)	Few directives on the existing loop; shared memory; no copying	One machine only	Good fit. The data fits in memory easily.
MPI	Works across many machines	Data has to be scattered and results gathered; message cost	Useful only for data that does not fit on one machine.
CUDA	Very high throughput on large arrays	Copying data to the GPU costs more than this small computation	Not suitable. The copy would take longer than the statistics.

Table 6: Comparison of parallel programming models

7.2 OpenMP constructs used

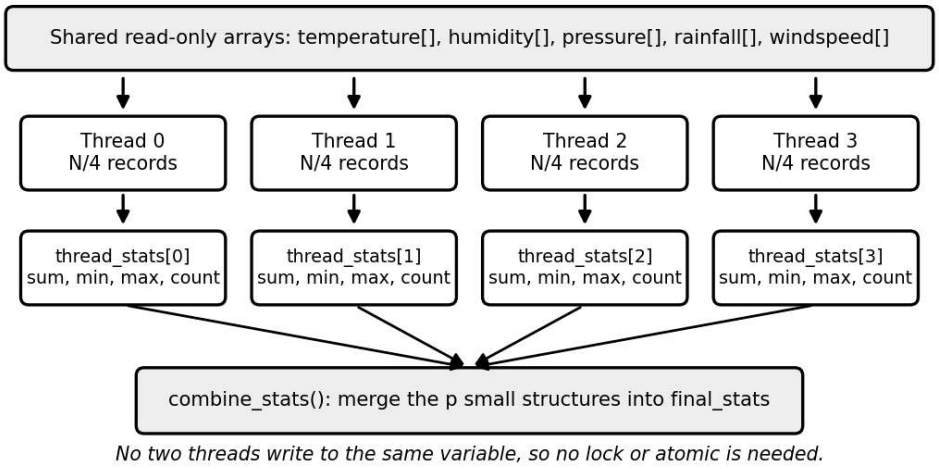


Figure 2: Data division among threads and the final merge

Construct	Purpose in this project
`#pragma omp parallel num_threads(p)`	Starts a team of p threads. The number p is given on the command line.
`omp_get_thread_num()`	Gives each thread its number, used to select `thread_stats[id]`.
`#pragma omp for schedule(static)`	Divides the record loop into p blocks of consecutive records.
private `WeatherStats` per thread	Each thread updates only its own structure, so no lock or atomic is used.
`combine_stats()` after the region	Merges the p structures one after another.
`omp_get_wtime()`	Wall-clock timer around the parallel region and the merge.

Static scheduling suits this loop because every record needs the same amount of work, so all threads finish at about the same time and there is no scheduling cost at run time.

7.3 Why there is no race condition

The data arrays are only read. The loop counter is private to each thread. Every write goes to `thread_stats[thread_id]`, which belongs to one thread only. The results are combined after the parallel region has ended, and the end of the region is an implicit barrier, so every partial result is complete by then. Counts, minimums and maximums do not depend on the order of merging. Sums can differ in the last digits when the order changes, which is discussed in Section 13.

8. Implementation

8.1 Updating the summary for one record

```
void process_record(WeatherStats *s, double temp, double hum,
                    double press, double rain, double wind)
{
   s->count++;
   s->temperature_sum += temp;
   if (temp < s->temperature_min) s->temperature_min = temp;
   if (temp > s->temperature_max) s->temperature_max = temp;
   /* the same three lines follow for humidity, pressure, rainfall and wind speed */
}
```

8.2 Parallel region

```
WeatherStats *thread_stats = malloc(num_threads * sizeof(WeatherStats));
for (int i = 0; i < num_threads; i++) initialize_stats(&thread_stats[i]);

double start_time = omp_get_wtime();
#pragma omp parallel num_threads(num_threads)
{
   WeatherStats *local_stats = &thread_stats[omp_get_thread_num()];
   #pragma omp for schedule(static)
   for (long long i = 0; i < count; i++)
      process_record(local_stats, temperature[i], humidity[i],
                     pressure[i], rainfall[i], windspeed[i]);
}
for (int i = 0; i < num_threads; i++)
   combine_stats(&final_stats, &thread_stats[i]);
double execution_time = omp_get_wtime() - start_time;
```

9. Dry Run / Example

We take the first 8 temperature values of the dataset and run the parallel algorithm with p = 2 threads and `schedule(static)`. Thread 0 gets records 0 to 3 and thread 1 gets records 4 to 7.

	Thread 0 (records 0–3)	Thread 1 (records 4–7)
Values (°C)	24.10, 24.10, 25.50, 28.20	29.60, 30.10, 30.20, 30.10
Private count	4	4
Private sum	101.90	120.00
Private minimum	24.10	29.60
Private maximum	28.20	30.20

Table 8: Partial results of the two threads

Merging gives count = 8, sum = 101.90 + 120.00 = 221.90, minimum = smaller of 24.10 and 29.60 = 24.10, maximum = larger of 28.20 and 30.20 = 30.20. The mean is 221.90 / 8 = 27.74 °C. The sequential loop over the same 8 values gives the same count, sum, minimum, maximum and mean.

10. Experimental Setup

Parameter	Value
Dataset	AWS Dataset Pangandaran, 8,085 records
Thread counts	p = 1, 2, 4 and 8
Compute time	Time of the statistics loop (and the merge in the parallel version). File reading is not included.
Total time	Time of the complete program run, measured from outside: start-up, reading, parsing and computing
Speedup	$S(p) = T_{seq} / T_{par}(p)$
Parallel efficiency	$E(p) = S(p) / p \times 100\ \%$

Table 9: Experimental parameters

If the machine used for all measurements has only one CPU core, OpenMP creates the threads, but they take turns on the same core, so a real speedup cannot appear. The measured thread results can therefore show how much it costs to use threads, and not how fast they can be on more cores. The programs, the input files and the benchmark script can also be run unchanged on a multi-core computer to get speedup figures.

11. Results and Performance Analysis

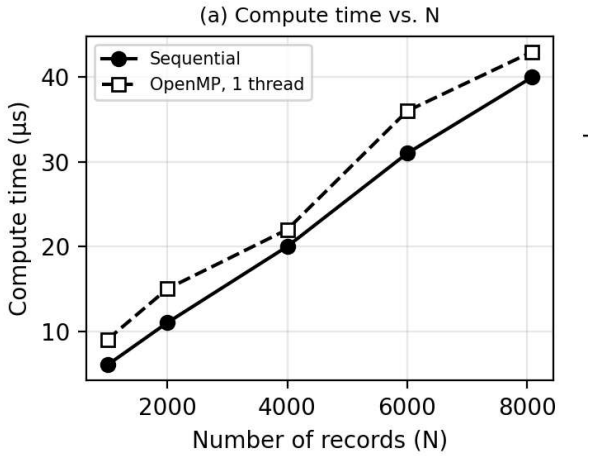


Figure 3: Compute time against number of records

The compute time grows in a straight line with N, as expected for a single pass over the data: it goes from about 6 µs at 1,000 records to 40 µs at 8,085 records. The cost is about 5 ns per record for every size. The smallest inputs cost slightly more per record, which is a fixed start-up cost of the loop. The timer `clock()` has a resolution of one microsecond, so the values for the smallest inputs are coarse. Individual runs at N = 8,085 ranged from 33 to 86 µs, which is why medians are used.

11.1 Where does the time go?

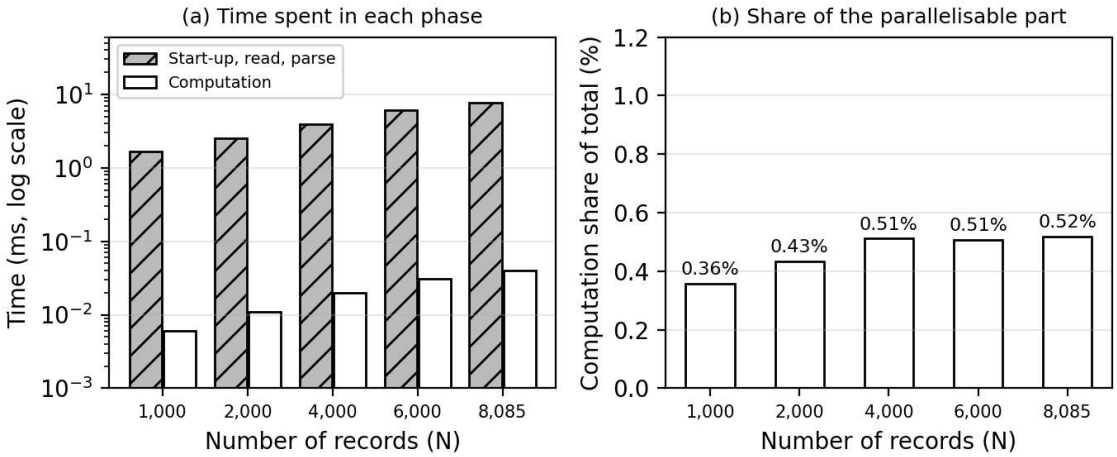


Figure 4: (a) Time spent in reading/parsing and in computing, (b) share of the computation

For the full dataset the complete run takes 7.7 ms, of which the statistics loop is only 40 µs, or 0.52 %. The remaining time is process start-up and the reading and parsing of the CSV text, about 0.95 µs per record. That is roughly 190 times the cost of the computation itself. The loop is the only part that is parallelised, so this ratio decides how much the parallel program can gain.

11.2 Effect of increasing the number of threads

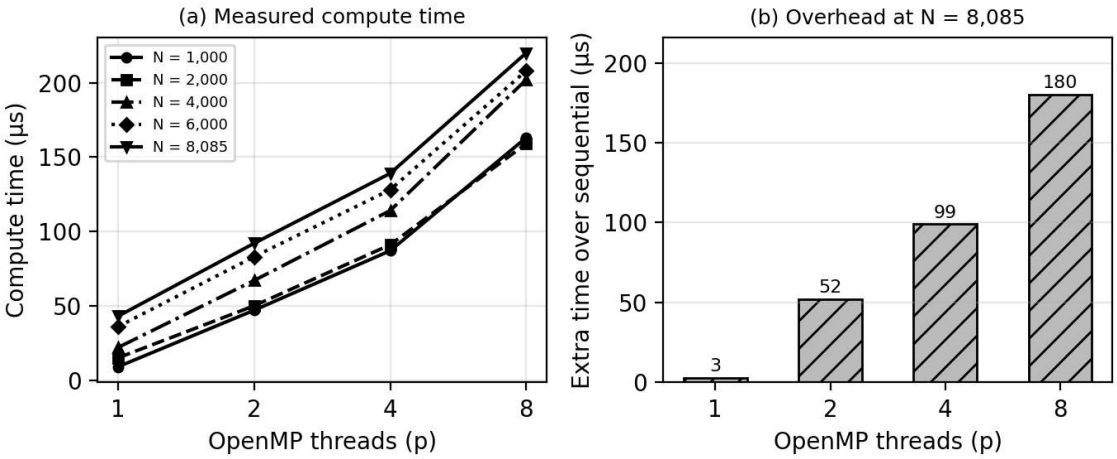


Figure 5: (a) Measured compute time against threads, (b) extra time compared with the sequential loop

Threads (p)	Compute time (µs)	Speedup S	Efficiency E
1	43	0.93	93 %
2	92	0.43	22 %
4	139	0.29	7 %
8	220	0.18	2 %

Table 10: Speedup and efficiency of the compute phase at N = 8,085 (one CPU core)

As expected for one core, the parallel version is not faster than the sequential one. The speedup is below 1 and the efficiency falls roughly as 1/p. The extra time over the sequential loop is about 50 µs for 2 threads, 100 µs for 4 threads and 180 µs for 8 threads, which is around 25 µs for every thread added. This is the cost of creating and joining the threads. With only 8,085 records the loop itself lasts 40 µs, so the overhead is larger than the work. At about 5 ns per record, a few thousand records per thread are needed just to pay for one thread, so parallelising this loop only makes sense for tens of thousands of records or more. We expect, but have not measured, that on a multi-core machine the loop would speed up once the input is that large, until memory bandwidth becomes the limit.

11.3 Bottlenecks

- Reading and parsing of the CSV file (`fgets` and `sscanf`) is sequential and takes more than 99 % of the run time. This is the main bottleneck.
- Thread start-up and joining costs about 25 µs per thread. For small inputs this is more than the useful work.
- The loop does very little arithmetic per record and reads 40 bytes per record, so on a multi-core machine its scaling will be limited by memory bandwidth for large inputs.

12. Correctness Verification

A parallel program is only useful if it gives the same answer as the sequential one. We compared the complete printed output of the two programs (count, averages, minimums, maximums and the rainfall total) on the full dataset for 1, 2, 3, 4, 7, 8 and 16 threads. Odd thread counts such as 3 and 7 were included on purpose because they do not divide 8,085 evenly, and 16 is more threads than there are cores.

Input	Threads tested	Printed output (2 decimals)	Difference at decimals
Full dataset, 8,085 records	1, 2, 3, 4, 7, 8, 16	Identical in all 7 runs	0 in all 7 runs

Table 12: Sequential output compared with OpenMP output

We also recomputed the statistics with a separate Python script that uses the csv module. It gave a mean wind speed of 4.4022 km/h and a rainfall total of 4758.9 mm, which matches the C programs (4.40 and 4758.90). Finally, the results were compared with the MIN, MAX and AVERAGE rows written by the weather station at the end of the file.

Statistic	Our result	Station summary row	Match
Temperature min / mean / max (°C)	20.80 / 25.55 / 33.00	20.8 / 25.55 / 33	Yes
Humidity min / mean / max (%)	52 / 84.06 / 99	52 / 84.06 / 99	Yes
Pressure min / mean / max (hPa)	1005.70 / 1011.82 / 1017.60	1005.7 / 1011.82 / 1017.6	Yes
Rainfall mean / max (mm)	0.59 / 38.10	0.59 / 38.10	Yes
Wind speed min / max (km/h)	0.00 / 23.40	0 / 23.4	Yes
Wind speed mean (km/h)	4.78	4.78	Yes

Table 13: Comparison with the summary rows of the data file

Counts, minimums and maximums cannot change with the thread count. Sums can differ in the last bits when threads add numbers in a different order, because floating-point addition is not exactly associative. For this dataset the difference was zero even when the results were printed with 10 decimal places.

13. Complexity Analysis

Let N be the number of records, p the number of threads and V the number of variables (V = 5, a constant).

Version	Time	Space
Sequential computation	$O(N \cdot V) = O(N)$	O(N) for the arrays, O(1) for the statistics
Parallel computation	$O(N / p + p)$	$O(N + p)$
Reading and parsing (both)	O(N), sequential	O(N)

Table 14: Complexity summary

The N/p term is the loop shared by the threads and the p term is the merge. The measured compute times (Figure 3) follow the O(N) behaviour. No algorithm can do better than O(N) for this problem, since every record has to be read at least once.

14. Advantages

- The method is simple: one private summary per thread and one merge at the end.
- There is no lock or atomic operation inside the loop, so there is no race condition.
- The number of threads is a command-line argument, so one program supports all thread-count experiments.
- The programs skip the header and the summary rows automatically.
- The browser dashboard gives a quick look at the same statistics.

15. Limitations

- Only the computing loop is parallel. Reading and parsing are sequential and make up more than 99 % of the run time, so the whole program does not become faster.
- The per-thread structures are not padded, so false sharing is possible on real hardware.
- The dashboard speedup table is an estimate with fixed constants and not a measurement.

16. Future Scope

- Parallelise the reading and parsing: load the file into memory, split it into blocks at line boundaries and let each thread parse its own block (for example with `strtod` instead of `sscanf`).
- Use `omp_get_wtime()` in the sequential program as well.
- Test with much larger datasets from many stations; use MPI if the data no longer fits on one machine.
- Add daily and monthly averages and trend detection.

17. Team Contributions

Member	Contribution
Akshita Pahade	Parallel.c: the thread-count argument, per-thread structures, the parallel region with schedule(static), and the merge step (combine_stats). Checked output across 1 to 16 threads.
Shalaka Parhad	Verified correctness of sequential and parallel execution. Calculated time and space complexities. Analysed the bottlenecks and limitation, and wrote the LLM usage log.
Parul Deshmukh	Dataset selection and Workflow analysis, selection of appropriate parallel programming APIs, prompting
Samruddhi Pashankar	WeatherStats struct, process_record, result printing and timing in sequential.c. Tested for race conditions in parallel programming. Calculated speedup and efficiency.

Table 15: Individual contributions

18. Conclusion

We built a sequential and an OpenMP version of a program that computes the mean, minimum, maximum and total of five weather variables, and a browser dashboard to display them. The parallel version keeps one private summary per thread and merges them after the loop, so it needs no locks and has no race condition. Its output matched the sequential output for every thread count tried, and the values agree with the station's own summary rows (the one difference, the mean wind speed, comes from calm readings being counted).

On the 8,085-record dataset the statistics loop takes about 40 µs, which is about half a percent of the whole run. Reading and parsing the CSV file takes the rest. By Amdahl's law this keeps the speedup of the whole program close to 1, however many cores are used.

19. References

[1] OpenMP Architecture Review Board, "OpenMP Application Programming Interface," openmp.org.

[2] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS Conference Proceedings, 1967.

[3] J. L. Gustafson, "Reevaluating Amdahl's Law," Communications of the ACM, 31(5), 1988.

[4] B. Chapman, G. Jost and R. van der Pas, Using OpenMP: Portable Shared Memory Parallel Programming, MIT Press, 2008.

[5] PCE Instruments, PCE-FWS 20N weather station, pce-instruments.com (source of the AWS data format).